```
!pip -q install tensorflow pandas numpy matplotlib scikit-learn
```

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# Load the downloaded dataset
file_path = (
    "/content/rainfall_data/"
    "Indian Rainfall Dataset District-wise Daily Measurements.csv"
)

df = pd.read_csv(file_path)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nFirst 5 rows:")
display(df.head())
```

```
Dataset loaded successfully!
Shape: (8790, 1)

Column names:
['state;"district";"month";"1st";"2nd";"3rd";"4th";"5th";"6th";"7th";"8th";"9th";"10th";

First 5 rows:
```

	state;"district";"month";"1st";"2nd";"3rd";"4th";"5th";"6th";"7th";"8th";"9th";"10th
0	
1	
2	
3	
4	

```
file_path = (
    "/content/rainfall_data/"
    "Indian Rainfall Dataset District-wise Daily Measurements.csv"
)

df = pd.read_csv(file_path, sep=";")

# Remove extra spaces and quotation marks from column names
df.columns = (
    df.columns.astype(str)
    .str.strip()
    .str.replace('"', '', regex=False)
)

print("Dataset shape:", df.shape)
```

```
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nFirst 5 rows:")
display(df.head())

print("\nAvailable districts:")
print(sorted(df["district"].dropna().unique()))
```

```
Dataset shape: (8790, 34)

Column names:
['state', 'district', 'month', '1st', '2nd', '3rd', '4th', '5th', '6th', '7th', '8th', '

First 5 rows:
```

	state	district	month	1st	2nd	3rd	4th	5th	6th	7th	...	22nd	23rd	
0	Andaman & Nicobar	Nicobars	1	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
1	Andaman & Nicobar	North And Middle Andaman	1	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
2	Andaman & Nicobar	South Andamans	1	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
3	Andhra Pradesh	Anantapur	1	0.379965	0.000000	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0
4	Andhra Pradesh	Chittoor	1	14.144900	0.857263	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	

5 rows × 34 columns

```
Available districts:
['24 Paraganas North', '24 Paraganas South', 'Adilabad', 'Agar Malwa', 'Agra', 'Ahamadna
```

```
districts = df["district"].astype(str).str.strip()

print("Coimbatore matches:")
print(
    df.loc[
        districts.str.contains("Coimbatore", case=False, na=False),
        ["state", "district", "month"]
    ].drop_duplicates().to_string(index=False)
)

print("\nTamil Nadu district names found:")
print(
    sorted(
        df.loc[
            df["state"].astype(str).str.contains("Tamil Nadu", case=False, na=False),
            "district"
        ].dropna().astype(str).unique()
    )
)
```

```
Coimbatore matches:
     state   district  month
Tamil Nadu Coimbatore      1
Tamil Nadu Coimbatore      2
Tamil Nadu Coimbatore      3
Tamil Nadu Coimbatore      4
```

```
Tamil Nadu Coimbatore      5
Tamil Nadu Coimbatore      6
Tamil Nadu Coimbatore      7
Tamil Nadu Coimbatore      8
Tamil Nadu Coimbatore      9
Tamil Nadu Coimbatore     10
Tamil Nadu Coimbatore     11
Tamil Nadu Coimbatore     12

Tamil Nadu district names found:
['Ariyalur', 'Chengalpattu', 'Chennai', 'Coimbatore', 'Cuddalore', 'Dharmapuri', 'Dindig
```

```
district_df = df[
    (df["state"].astype(str).str.strip().str.lower() == "tamil nadu") &
    (df["district"].astype(str).str.strip().str.lower() == "coimbatore")
].copy()

print("Selected rows:", len(district_df))
display(district_df[["state", "district", "month"]].head())
```

```
Selected rows: 12
```

	state	district	month
546	Tamil Nadu	Coimbatore	1
1278	Tamil Nadu	Coimbatore	2
2010	Tamil Nadu	Coimbatore	3
2742	Tamil Nadu	Coimbatore	4
3474	Tamil Nadu	Coimbatore	5

```
import re

day_cols = [
    col for col in district_df.columns
    if re.fullmatch(r"\d+(st|nd|rd|th)", str(col).strip().lower())
]

daily_df = district_df.melt(
    id_vars=["state", "district", "month"],
    value_vars=day_cols,
    var_name="day",
    value_name="rainfall"
)

daily_df["day"] = daily_df["day"].astype(str).str.extract(r"(\d+)").astype(int)
daily_df["month"] = pd.to_numeric(daily_df["month"], errors="coerce")
daily_df["rainfall"] = pd.to_numeric(daily_df["rainfall"], errors="coerce")

# Treat negative rainfall values as missing
daily_df.loc[daily_df["rainfall"] < 0, "rainfall"] = np.nan

daily_df["date"] = pd.to_datetime(
    {
        "year": 2022,
        "month": daily_df["month"],
        "day": daily_df["day"]
    },
    errors="coerce"
)

daily_df = daily_df.dropna(subset=["date"])
daily_df = daily_df.groupby("date")["rainfall"].mean().sort_index().to_frame()
```

```
# Include all dates in 2022
all_dates = pd.date_range("2022-01-01", "2022-12-31", freq="D")
daily_df = daily_df.reindex(all_dates)
daily_df.index.name = "date"

# Fill missing rainfall measurements
daily_df["rainfall"] = daily_df["rainfall"].interpolate(method="time")
daily_df["rainfall"] = daily_df["rainfall"].ffill().bfill()

print("Daily records:", len(daily_df))
print("Missing values:", daily_df["rainfall"].isna().sum())
display(daily_df.head(10))
```

```
Daily records: 365
Missing values: 0
```

	rainfall
date	
2022-01-01	2.148890
2022-01-02	9.110140
2022-01-03	0.046436
2022-01-04	0.000000
2022-01-05	0.000000
2022-01-06	0.000000
2022-01-07	0.000000
2022-01-08	0.026535
2022-01-09	0.006692
2022-01-10	0.000000

```
# Extract rainfall values
rain_values = daily_df["rainfall"].values.reshape(-1, 1)

# Chronological 80% training and 20% testing split
split_index = int(len(rain_values) * 0.80)

# Normalize using training data only
scaler = MinMaxScaler()
scaler.fit(rain_values[:split_index])
scaled_values = scaler.transform(rain_values)

# Create sequences: previous 7 days -> next day's rainfall
X, y, target_indices = [], [], []

for i in range(7, len(scaled_values)):
    X.append(scaled_values[i-7:i])
    y.append(scaled_values[i])
    target_indices.append(i)

X = np.array(X)
y = np.array(y)
target_indices = np.array(target_indices)

# Split sequences chronologically
train_mask = target_indices < split_index
test_mask = target_indices >= split_index

X_train, y_train = X[train_mask], y[train_mask]
X_test, y_test = X[test_mask], y[test_mask]
```

```
print("Training inputs:", X_train.shape)
print("Training targets:", y_train.shape)
print("Testing inputs:", X_test.shape)
print("Testing targets:", y_test.shape)
```

```
Training inputs: (285, 7, 1)
Training targets: (285, 1)
Testing inputs: (73, 7, 1)
Testing targets: (73, 1)
```

```
model = Sequential([
    LSTM(32, input_shape=(7, 1)),
    Dropout(0.2),
    Dense(16, activation="relu"),
    Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=10,
    restore_best_weights=True
)

history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=16,
    validation_split=0.15,
    shuffle=False,
    callbacks=[early_stop],
    verbose=1
)
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do
  super().__init__(**kwargs)
Epoch 1/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 4s 25ms/step - loss: 0.0273 - val_loss: 0.0078
Epoch 2/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0191 - val_loss: 0.0087
Epoch 3/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0177 - val_loss: 0.0103
Epoch 4/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0176 - val_loss: 0.0094
Epoch 5/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0162 - val_loss: 0.0087
Epoch 6/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0154 - val_loss: 0.0085
Epoch 7/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0145 - val_loss: 0.0087
Epoch 8/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0141 - val_loss: 0.0085
Epoch 9/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0141 - val_loss: 0.0086
Epoch 10/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0136 - val_loss: 0.0089
Epoch 11/100
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0135 - val_loss: 0.0089
```

```
pred_scaled = model.predict(X_test)

actual = scaler.inverse_transform(
```

```
    y_test.reshape(-1, 1)
).flatten()

predicted = scaler.inverse_transform(
    pred_scaled
).flatten()

predicted = np.maximum(predicted, 0)

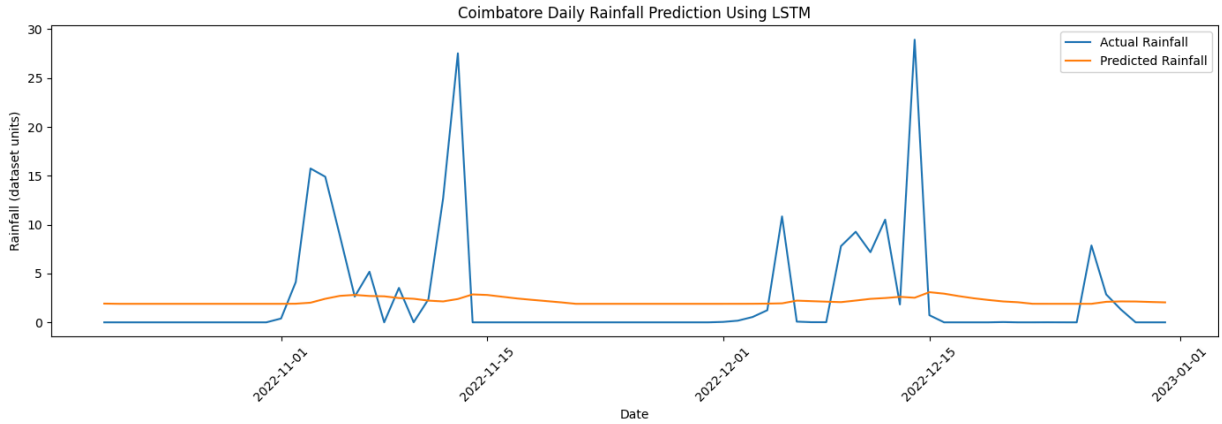
mae = mean_absolute_error(actual, predicted)
rmse = np.sqrt(mean_squared_error(actual, predicted))

print(f"MAE: {mae:.4f}")
print(f"RMSE: {rmse:.4f}")
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 242ms/step
MAE: 3.4476
RMSE: 5.6745
```

```
test_dates = daily_df.index[target_indices[test_mask]]

plt.figure(figsize=(14, 5))
plt.plot(test_dates, actual, label="Actual Rainfall")
plt.plot(test_dates, predicted, label="Predicted Rainfall")
plt.title("Coimbatore Daily Rainfall Prediction Using LSTM")
plt.xlabel("Date")
plt.ylabel("Rainfall (dataset units)")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```




```
current_sequence = scaled_values[-7:].reshape(1, 7, 1).copy()
future_scaled = []

for _ in range(7):
    next_value = model.predict(current_sequence, verbose=0)[0, 0]
    future_scaled.append(next_value)
```

```
    current_sequence = np.concatenate(
        [
            current_sequence[:, 1:, :],
            np.array(next_value).reshape(1, 1, 1)
        ],
        axis=1
    )

future_rainfall = scaler.inverse_transform(
    np.array(future_scaled).reshape(-1, 1)
).flatten()

future_rainfall = np.maximum(future_rainfall, 0)

future_dates = pd.date_range("2023-01-01", periods=7, freq="D")

forecast_df = pd.DataFrame({
    "Date": future_dates,
    "Predicted Rainfall": future_rainfall
})

display(forecast_df)
```

	Date	Predicted Rainfall
0	2023-01-01	2.006528
1	2023-01-02	2.026523
2	2023-01-03	2.018188
3	2023-01-04	2.035685
4	2023-01-05	2.057319
5	2023-01-06	2.079595
6	2023-01-07	2.096480